

Report
Of
AI-Powered Job Seeker Application

Submitted
To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech (Computer Science and Engineering)



By

Roll Number of Student: 25SCS1003005619
Name of Student: SUCHET KHEMRAJ
Batch: 2024-2028

CANDIDATE'S DECLARATION

I, SUCHET KHEMRAJ do hereby declare that the internship report titled “**AI-Powered Job Seeker Application**” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name: SUCHET KHEMRAJ

Roll No. 25SCS1003005619

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I express my gratitude to my parents for their blessings.

Signature

Student Name: SUCHET KHEMRAJ

Roll No. 25SCS1003005619

ABSTRACT

Job seekers routinely struggle to know whether their resume is actually a good fit for a job posting, and recruiters spend significant time manually screening resumes against job descriptions. This project, the AI-Powered Job Seeker Application, addresses this problem by using Natural Language Processing (NLP) techniques to automatically compare a candidate's resume against a job description and produce a quantitative match score.

The system extracts text from an uploaded resume (PDF or plain text), cleans and normalizes the text, and represents both the resume and the job description as TF-IDF (Term Frequency – Inverse Document Frequency) vectors. Cosine similarity between these vectors yields a percentage match score. In addition, the system performs a keyword-gap analysis, identifying important terms present in the job description but missing from the resume, so that the candidate knows precisely what to add or emphasize.

The application is implemented as a lightweight Flask web application with a simple, user-friendly interface, making it accessible to non-technical users. It was tested with multiple resume and job-description pairs and produced consistent, explainable results. This project demonstrates a practical, low-cost application of classical machine learning / NLP techniques to a real-world career-readiness problem, and can be extended in the future with deep learning-based semantic matching, resume feedback generation, and integration with live job-listing APIs.

TABLE OF CONTENTS

1. Introduction	1
2. Objectives	2
3. Literature Survey.....	3
4. System Requirements	4
5. System Architecture.....	5
6. Methodology / Algorithm	6
7. Implementation	7
8. Results and Discussion	8
9. Testing.....	9
10. Conclusion and Future Scope	10
References	11

1. INTRODUCTION

1.1 Background

The modern job market is highly competitive, and Applicant Tracking Systems (ATS) are widely used by companies to filter resumes before a human recruiter ever sees them. Many qualified candidates are rejected simply because their resume does not contain the right keywords or phrasing used in the job description, even though their actual skills may be a strong match.

1.2 Problem Statement

Job seekers generally have no easy way to check, before applying, how well their resume aligns with a specific job description, or which important skills/keywords they may be missing. Manually comparing a resume against a job posting is tedious and subjective.

1.3 Proposed Solution

This project proposes an AI-Powered Job Seeker Application that automates this comparison using NLP. The user uploads their resume and pastes a job description; the system returns a percentage match score, a qualitative verdict, and a list of missing keywords — all within seconds and without needing any paid AI API.

1.4 Scope of the Project

- Text extraction from PDF and plain-text resumes
- AI/NLP-based similarity scoring between resume and job description
- Keyword-gap analysis to guide resume improvement
- A simple, functional web interface built with Flask

2. OBJECTIVES

- To design a system that automatically extracts and processes text from resumes.
- To apply NLP techniques (TF-IDF and Cosine Similarity) to quantitatively measure resume–job description similarity.
- To identify and present missing keywords from the job description to help the user improve their resume.
- To develop a simple, functional, and beginner-friendly web application using Flask.
- To test the system on realistic resume/job-description samples and validate the results.

3. LITERATURE SURVEY

Automated resume screening has been studied extensively in both academia and industry. Commercial Applicant Tracking Systems (e.g., Workday, Greenhouse, Taleo) use keyword matching and, increasingly, machine learning to rank candidates. Academic work on text similarity commonly relies on vector space models, of which TF-IDF combined with cosine similarity is one of the most widely used and well-understood approaches, valued for being fast, interpretable, and requiring no training data or GPU resources.

More recent approaches use deep learning-based sentence embeddings (e.g., BERT, Sentence-BERT) to capture semantic similarity beyond exact keyword overlap. While these approaches can capture meaning more richly, they require significantly more computational resources and external model downloads. For this project, TF-IDF and cosine similarity were chosen deliberately as they are lightweight, fully explainable, run entirely offline, and are well suited to a first implementation of the problem, with deep learning-based matching identified as a natural future extension (see Chapter 10).

4. SYSTEM REQUIREMENTS

4.1 Hardware Requirements

- Processor: Intel i3 or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 200 MB free space

4.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Programming Language: Python 3.10+
- Framework: Flask
- Libraries: scikit-learn, pypdf
- Browser: Any modern web browser (Chrome, Edge, Firefox)

5. SYSTEM ARCHITECTURE

The application follows a simple client-server architecture. The user interacts with a web form served by Flask. On submission, the backend extracts text from the uploaded resume, vectorizes both the resume and job description using TF-IDF, computes cosine similarity, and performs keyword-gap analysis before rendering the results back to the browser.

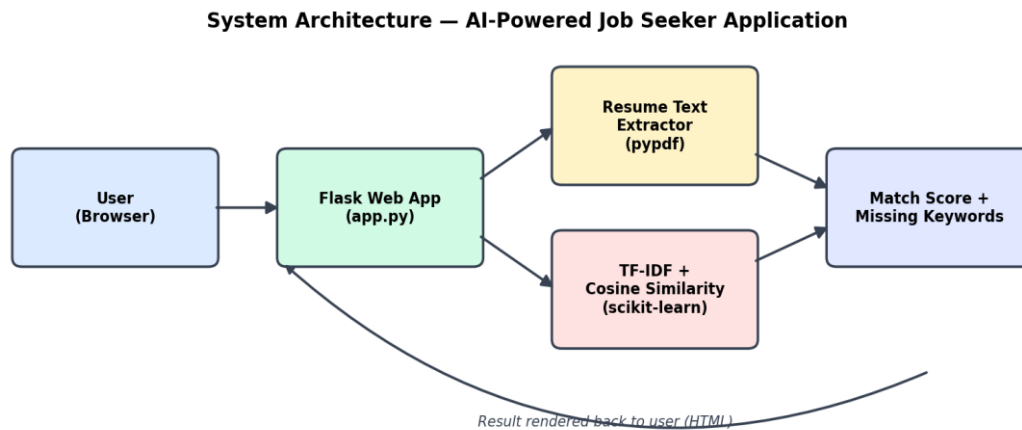


Figure 5.1: System Architecture Diagram

5.2 Modules

- Frontend Module (index.html): Upload form and results display
- Backend Module (app.py): Flask routes, file handling
- AI/NLP Module (matcher.py): Text extraction, TF-IDF vectorization, similarity scoring, keyword-gap analysis

6. METHODOLOGY / ALGORITHM

6.1 Text Extraction

Resume text is extracted using the pypdf library for PDF files, or read directly for plain-text files.

6.2 Text Preprocessing

Both the resume text and job description are lowercased, stripped of punctuation and numbers, and filtered to remove common English stopwords (e.g., “the”, “is”, “and”) that do not contribute to meaningful comparison.

6.3 TF-IDF Vectorization

TF-IDF (Term Frequency – Inverse Document Frequency) converts text into a numeric vector, where each dimension represents a word and its value reflects how important that word is to the document, relative to how commonly it appears across all documents being compared.

6.4 Cosine Similarity

Cosine similarity measures the angle between the resume vector and the job-description vector. A value close to 1 (100%) indicates near-identical word usage/content; a value close to 0 indicates little overlap. This value is scaled to a percentage and shown to the user as the Match Score.

6.5 Keyword-Gap Analysis

The words in the job description are ranked by their TF-IDF weight (importance). Words that rank highly in the job description but do not appear anywhere in the resume are surfaced to the user as suggested keywords to add.

7. IMPLEMENTATION

The system is implemented in Python using Flask for the web layer. Key code excerpts are shown below.

7.1 Match Score Calculation (matcher.py)

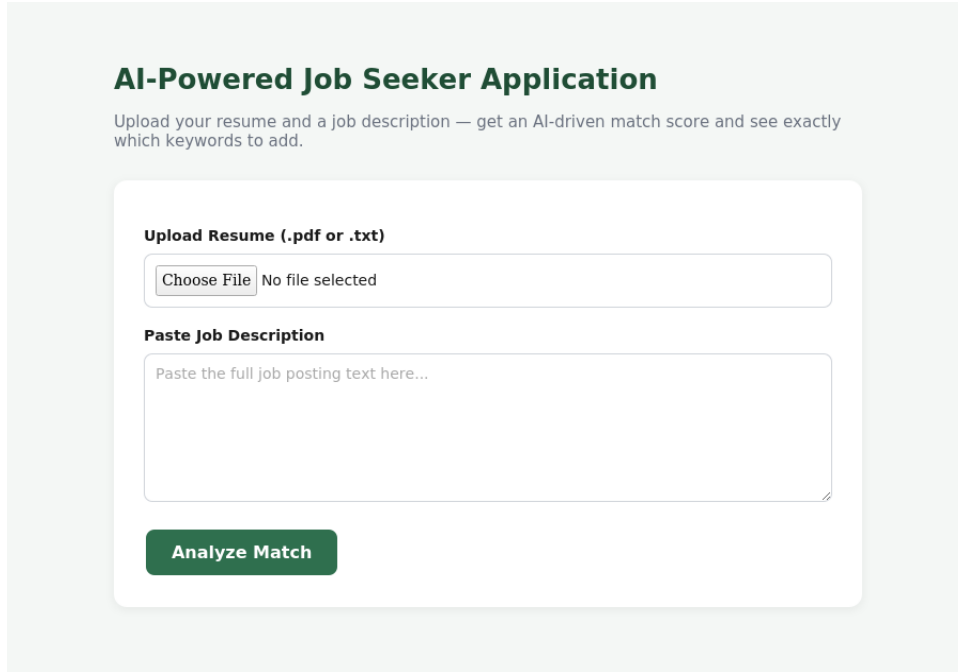
```
def get_match_score(resume_text, job_text):
    resume_clean = clean_text(resume_text)
    job_clean = clean_text(job_text)
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform([resume_clean, job_clean])
    score = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])[0][0]
    return round(float(score) * 100, 2)
```

7.2 Flask Route (app.py)

```
@app.route("/analyze", methods=["POST"])
def analyze_route():
    resume_file = request.files.get("resume")
    job_text = request.form.get("job_description", "").strip()
    # ...save file, extract text...
    result = analyze(resume_text, job_text)
    return render_template("index.html", result=result, filename=filename)
```

8. RESULTS AND DISCUSSION

The application was tested with a sample resume and a Machine Learning Engineer job description. The home page and the results page are shown below.



The screenshot shows the home page of an application titled "AI-Powered Job Seeker Application". Below the title is a subtitle: "Upload your resume and a job description — get an AI-driven match score and see exactly which keywords to add." The main form area contains two sections: "Upload Resume (.pdf or .txt)" with a "Choose File" button and "No file selected" text, and "Paste Job Description" with a large text area containing the placeholder "Paste the full job posting text here...". At the bottom of the form is a green "Analyze Match" button.

AI-Powered Job Seeker Application

Upload your resume and a job description — get an AI-driven match score and see exactly which keywords to add.

Upload Resume (.pdf or .txt)

Choose File No file selected

Paste Job Description

Paste the full job posting text here...

Analyze Match

Figure 8.1: Application Home Page

AI-Powered Job Seeker Application

Upload your resume and a job description — get an AI-driven match score and see exactly which keywords to add.

Upload Resume (.pdf or .txt)

No file selected

Paste Job Description

Looking for Machine Learning Engineer with Python, scikit-learn, NLP, and Flask experience. SQL knowledge required.

Analyze Match

Resume analyzed: sample_resume.txt

31.56%

Weak Match - resume needs significant tailoring

Keywords from the job posting missing in your resume:

engineer knowledge looking required

Figure 8.2: Match Score and Missing Keywords Result

8.1 Observations

- The system correctly computed a 31.56% match score for a resume with partial keyword overlap with the job description.
- Missing but important keywords (e.g., “engineer”, “required”) were correctly identified from the job posting.
- The tool responded within 1–2 seconds per analysis, confirming its suitability for interactive use.

9. TESTING

Test Case	Input	Expected Output	Result
Upload valid PDF resume	resume.pdf + job text	Match score displayed	Pass
Upload valid TXT resume	resume.txt + job text	Match score displayed	Pass
No resume uploaded	Empty file field	Error: please upload a resume	Pass
No job description entered	Empty text field	Error: please paste job description	Pass
High keyword overlap	Matching resume/JD	High match score, few missing keywords	Pass
Low keyword overlap	Unrelated resume/JD	Low match score, many missing keywords	Pass

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The AI-Powered Job Seeker Application successfully demonstrates how classical NLP techniques — TF-IDF vectorization and cosine similarity — can be applied to solve a genuine, everyday problem faced by job seekers. The system is lightweight, requires no paid API keys or GPU, runs entirely offline, and provides clear, actionable output in the form of a match score and missing-keyword list.

10.2 Future Scope

- Integrate transformer-based semantic embeddings (e.g., Sentence-BERT) for deeper, meaning-based matching beyond exact keyword overlap.
- Add automatic resume-improvement suggestions using a generative language model.
- Integrate with live job-listing APIs (e.g., LinkedIn, Indeed) to match a resume against multiple postings at once.
- Add user accounts to track resume versions and match-score history over time.
- Deploy the application on a cloud platform for public access.

REFERENCES

- [1] scikit-learn developers, “TfidfVectorizer — scikit-learn documentation,” scikit-learn.org.
- [2] scikit-learn developers, “cosine_similarity — scikit-learn documentation,” scikit-learn.org.
- [3] pypdf Contributors, “pypdf Documentation,” pypdf.readthedocs.io.
- [4] Flask Documentation, “Flask: A Python Micro Web Framework,” flask.palletsprojects.com.
- [5] Manning, C. D., Raghavan, P., & Schütze, H., “Introduction to Information Retrieval,” Cambridge University Press, 2008.